
VIP – AI Hub for Config-Driven Agent Lifecycle Management, Orchestration, and Observability

Je valide le dépôt du rapport PFE relatif à l'étudiant nommé ci-dessous / I
validate the submission of the student's report:

- Nom & Prénom /Name & Surname : Ibtihel Mnaja

Encadrant Entreprise/ Business site Supervisor

- Nom & Prénom /Name & Surname : Ahmed Rebai

Cachet & Signature / Stamp & Signature

Encadrant Académique/Academic Supervisor

- Nom & Prénom /Name & Surname : Jihen Jebri

Signature / Signature

Ce formulaire doit être rempli, signé et **scanné**/This form must be completed, signed and **scanned**.

Ce formulaire doit être introduit après la page de garde/ This form must be inserted after the cover page.



ESPRIT SCHOOL OF ENGINEERING

www.esprit.tn - E-mail : contact@esprit.tn

Siège Social : 18 rue de l'Usine - Charguia II - 2035 - Tél. : +216 71 941 541 - Fax. : +216 71 941 889

Annexe : Z.I. Chotrana II - B.P. 160 - 2083 - Pôle Technologique - El Ghazala - Tél. : +216 70 685 685 - Fax. : +216 70 685 454

2025 - 2026
GRADUATION PROJECT

NATIONAL ENGINEERING DEGREE

SPECIALTY : Data Science

**VIP — Agentic AI Hub for Config-Driven
Agent Lifecycle Management and
Orchestration**

By: *Ibtihel Mnaja*

Academic supervisor: *Jihen Jebri*

Corporate Internship Supervisor: *Ahmed Rebai*

VALUE

Abstract

Agentic Artificial Intelligence refers to AI systems capable of performing tasks through autonomous or semi-autonomous agents that can reason, use tools, interact with external systems, and collaborate within structured workflows. In the financial services industry, the adoption of such systems requires not only advanced AI capabilities, but also strong guarantees in terms of scalability, control, auditability, and operational reliability.

This internship report presents the design and implementation of an **AI Agentic Hub Platform** aimed at building, deploying, and operating intelligent applications at scale. The proposed platform is based on a modular multi-agent architecture in which agents are generated from declarative configurations, deployed as independent containerized services, and coordinated through an orchestration layer. This approach makes it possible to create specialized agents without modifying their source code, by relying on configurable prompts, tools, model parameters, and input/output schemas.

The work focuses on several key aspects of enterprise AI deployment, including agent generation, workflow orchestration, schema-based communication, prompt and artifact versioning using MLflow, Docker-based containerization, and observability. The objective is to provide a controlled and extensible blueprint for developing financial AI applications while reducing fragmentation, improving governance, and ensuring traceability across the full lifecycle of intelligent systems.

Keywords: Agentic AI, Multi-Agent Systems, AI Orchestration, Docker, MLflow, Financial Services, Schema Validation, Observability.

Summary

This internship project focuses on the design and implementation of **VIP – Value Intelligent Platform**, an **Enterprise-Grade Agentic AI Hub** aimed at building, deploying, and operating intelligent applications at scale.

- **Dynamic Agent Generation:** Developed a configurable Agent Factory capable of creating specialized AI agents from declarative configurations, without modifying the source code.
- **Modular and Containerized Architecture:** Designed agents as independent FastAPI services deployed in isolated Docker containers, ensuring scalability, separation of concerns, and easier maintenance.
- **Centralized Governance and Versioning:** Integrated MLflow to manage prompts, schemas, and configuration artifacts, enabling traceability, version control, and a “Zero Code Change” approach.
- **Intelligent Orchestration:** Implemented an orchestration layer supported by an adapter to coordinate multi-agent workflows and manage schema transformations between agents.

Conclusion: The platform provides a scalable and reusable blueprint for enterprise AI applications, allowing organizations to transform complex business processes into modular, auditable, and intelligent agent-based workflows.

Acknowledgments

I would first like to express my sincere gratitude to the entire team at **Value Digital Services** for welcoming me into such a professional, collaborative, and innovative environment. The experience gained throughout this internship has been both enriching and inspiring, and I truly appreciated the support and positive atmosphere within the company.

I am deeply grateful to my professional supervisor, Mr. **Ahmed REBAI**, for his guidance, technical expertise, and continuous support throughout this project. His vision, advice, and constructive feedback played a major role in the successful realization of the *VIP – Value Intelligent Platform*.

I would also like to sincerely thank my academic supervisor, Ms. **Jihen JERBI**, for her valuable supervision, encouragement, and academic guidance. Her support helped ensure the quality and rigor of this work throughout the internship period.

On a personal note, I would like to express my deepest gratitude to my parents, **Hayet** and **Ali**, for their unconditional love, sacrifices, and constant encouragement throughout my studies. Their trust and support have always been my greatest source of motivation.

I would also like to warmly thank my sister **Dhouha**, as well as my brothers **Borhen** and **Louey**, for their continuous encouragement, patience, and support during both the challenging and rewarding moments of this journey.

Finally, I would like to thank everyone who contributed, directly or indirectly, to the success of this internship experience and to the completion of this project. This experience has been a major step in both my academic and professional journey, and I am truly grateful for the knowledge and experience I have gained.

Table of Contents

Glossary	XI
Introduction	1
1 Introduction and Project Context	2
1.1 Introduction	2
1.2 Host Company and Areas of Expertise	2
1.3 Project Overview	3
1.4 Problem Statement	3
1.5 Objectives	4
1.6 Technology Choices	5
1.6.1 Development Frameworks and Dev Environment	5
1.6.2 LLM and SLM Providers	6
1.6.3 Database, Storage, and Experiment Tracking	7
1.6.4 Deployment and Containerization	7
1.6.5 Observability and Monitoring	8
1.7 Methodology	8
1.7.1 Methodological Approaches Considered	8
1.7.2 Scrum Agile Methodology	8
1.7.3 IBM Data Science Methodology – The Foundational Methodology	8
1.7.4 IBM Agent Development Lifecycle	10
1.7.5 Comparative Analysis of the Methodologies	11
1.7.6 Adopted Hybrid Methodology for the VIP Project	12
1.8 Project Timeline	13
1.9 Summary	13
2 State of the Art	15
2.1 AI Agents and Orchestration	15
2.1.1 Definitions	15
2.1.2 LLM and SLM Overview	16
2.1.3 Motivation for Orchestration Frameworks	16
2.1.4 Framework Survey	16
2.2 Comparison of Existing Platforms	17
2.3 LLM and SLM Providers	19
2.4 Evaluation Framework	19
2.5 Gap Analysis and VIP Positioning	20
2.6 Summary and Transition	21

List of Figures

1	Logo of Value Digital Services	2
2	Logo of FastAPI	5
3	Logo of Next.js	5
4	Logo of LangChain	6
5	Logo of LangGraph	6
6	Logo of Jupyter Notebook	6
7	Logo of OpenAI	6
8	Logo of Groq	6
9	Logo of Microsoft Phi-4	6
10	Logo of Meta Llama	7
11	Logo of PostgreSQL	7
12	Logo of MLflow	7
13	Logo of Docker	7
14	Logo of Docker Compose	8
15	Logo of Laminar	8
16	Original IBM Data Science Methodology lifecycle	10
17	IBM Agent Development Lifecycle (ADLC)	11
18	Hybrid methodology adopted for the VIP project	13
2.1	Orchestration patterns diagram	15
2.2	Visual workflow orchestration model—n8n-style node-based workflow.	18
2.3	Role-based crew coordination—CrewAI-style manager/workers interaction.	18
2.4	Managed operational platform lifecycle	18
2.5	Evaluation pipeline—golden dataset, generation, metrics, judge, logging.	20
2.6	VIP gap positioning matrix—existing tools vs VIP capabilities.	21

List of Tables

1	Comparison of the methodological approaches considered for the VIP project .	12
2	Comparison of existing platforms	19
3	Provider comparison	19

Glossary

<i>ADLC:</i>	Agent Development Lifecycle
<i>AI:</i>	Artificial Intelligence
<i>API:</i>	Application Programming Interface
<i>IBM:</i>	International Business Machines
<i>LLM:</i>	Large Language Model
<i>QA:</i>	Quality Assurance
<i>REST:</i>	Representational State Transfer
<i>UI:</i>	User Interface
<i>UX:</i>	User Experience
<i>VIP:</i>	Value Intelligent Platform

Introduction

The recent evolution of Artificial Intelligence has marked a major shift from traditional conversational models toward *agentic systems*. These systems are characterized by their ability to reason, use external tools, interact with data sources, and execute complex multi-step workflows with a certain level of autonomy. In this context, the subject of this internship focuses on the design and implementation of **VIP – Value Intelligent Platform**, an enterprise-grade Agentic AI Hub intended to build, deploy, and operate intelligent applications at scale across different business domains.

Despite the significant progress of Large Language Models, their integration into enterprise environments remains challenging. The main difficulty lies in transforming AI prototypes into reliable, reusable, and governable systems that can be deployed in real operational contexts. Traditional approaches often rely on hard-coded agents, making them difficult to adapt, maintain, or extend. The problem addressed in this project is therefore to design a modular platform capable of generating specialized agents from declarative configurations, while ensuring scalability, traceability, and operational control.

The adopted approach is based on a decoupled and containerized multi-agent architecture. Each agent is designed as an independent FastAPI service, deployed in a Docker container, and configured through prompts, tools, model parameters, and input/output schemas. MLflow is used to manage and version configuration artifacts such as prompts and schemas, while an orchestration layer coordinates the execution of agent workflows. A semantic adapter is also introduced to manage schema transformations between agents and ensure smooth communication across the platform.

This report is organized into five chapters detailing the progression of the work carried out during the internship. **Chapter 1** presents the general context of the internship and the host company. **Chapter 2** introduces the preliminary work completed during the onboarding phase. **Chapter 3** presents the overall architecture of the VIP platform. **Chapter 4** details the design and implementation of the agent generation mechanism and the core platform components. **Chapter 5** focuses on orchestration, observability, and evaluation aspects. Finally, a **conclusion** summarizes the main contributions of the project and proposes possible future improvements.

Chapter 1

Introduction and Project Context

1.1 Introduction

The rapid adoption of AI agents and generative language models, including large language models (LLMs) and small language models (SLMs), is creating new opportunities for enterprise automation while increasing the need for reliable orchestration, deployment, and observability. In this context, my internship at **Value Digital Services** focused on building a modular, config-driven platform for agent generation and management, alongside the evaluation of small language models through SLM-evals. This chapter introduces the host company, clarifies the project goals and problem context, and presents the methodology followed throughout the internship.

1.2 Host Company and Areas of Expertise

Value Digital Services is a Tunisian company founded in 2019 and specialized in digital transformation and AI consulting. Its activities combine software engineering, data engineering, artificial intelligence, and strategic consulting, which made it an appropriate environment for an internship focused on building an enterprise AI agent platform and evaluating small language models.



Figure 1: Logo of Value Digital Services

- **Digital Factory:** software engineers, DevOps engineers, QA engineers, business analysts, and UX/UI designers build and deliver modern digital products.
- **Data Factory:** data engineers and data analysts design data pipelines, analytics workflows, and data platforms that support decision-making.

- **AI Factory:** AI researchers and machine learning engineers design and deploy AI-driven solutions, including generative AI and intelligent automation systems.
- **Advisory:** consultants support clients in structuring digital transformation initiatives, aligning technical choices with business and governance needs.

These poles work together to deliver modern digital solutions, from consulting and design to data infrastructure, AI implementation, deployment, and operational support.

1.3 Project Overview

The project, entitled **VIP – Value Intelligent Platform**, aims to design and implement an enterprise-grade AI agent orchestration platform capable of building, deploying, and managing intelligent applications through a modular multi-agent architecture. Unlike traditional monolithic AI systems, the platform follows an *Agent-as-a-Service* approach, where specialized agents are dynamically generated from declarative configurations and deployed as independent containerized services.

VIP provides a unified environment for agent creation, workflow orchestration, observability, and lifecycle management. Users can configure agents by selecting models, prompts, schemas, providers, and tools without directly modifying the source code. The generated agents can then be integrated into multi-agent execution pipelines coordinated through an orchestration layer and monitored through built-in observability services.

This platform directly addresses the gaps identified in existing solutions by combining support for multiple LLM providers and open-source models with a modular architecture, declarative configuration management, and built-in observability. This design reduces dependency on a single proprietary provider, simplifies agent creation and deployment, and makes agent behavior easier to trace, evaluate, and improve over time.

Alongside VIP, I developed the **SLM-evals** side project, a reproducible evaluation pipeline for benchmarking LLM and SLM providers. It compares OpenAI, Groq, Phi-4, Llama 3.1, and other alternatives under controlled prompts and golden datasets, while tracking latency, token usage, and layered evaluation metrics. The results of this pipeline guide the choice of models for the *LLM-provider* layer and support VIP’s objective of maintaining provider flexibility while selecting models that fit performance, cost, and reliability requirements.

1.4 Problem Statement

Despite the rapid evolution of AI agents and large language models, the deployment of agentic systems in enterprise environments remains complex. These challenges matter because enterprises need systems that can scale reliably, remain auditable, control operational costs, and comply with internal governance and security requirements. When agent platforms lack provider flexibility, observability, automation, or modular orchestration, they become difficult to deploy at scale and hard to maintain in production.

The main challenges addressed by this project can be summarized as follows:

- **High Operational Costs and Vendor Lock-in:** Existing AI agent solutions can be expensive to operate at scale, especially when they depend heavily on proprietary LLM APIs. This dependence increases costs, limits provider flexibility, and makes it harder to switch between OpenAI, Groq, Azure, open-source LLMs, or SLM alternatives.
- **Poor Observability:** Many current systems lack sufficient traceability, monitoring, and execution tracking. This limits debugging, performance evaluation, and the audit evidence required for compliance and governance.
- **Absence of Automated Deployment:** Creating and deploying new agents is often a manual and time-consuming process. Without automated deployment mechanisms, it becomes difficult to rapidly generate, configure, and launch agents in a consistent way.
- **Lack of Orchestration and Modularity:** Existing approaches often do not provide strong workflow orchestration capabilities. Agents are not always designed as reusable and modular components, which limits workflow composition and makes multi-agent collaboration harder to manage.

Based on these limitations, the central problem of this project can be formulated as follows:

How can we design a modular, secure, observable, and cost-optimized AI agent orchestration platform that enables dynamic agent creation, automated deployment, and workflow composition while evaluating and integrating multiple LLM and SLM providers to reduce vendor lock-in, control costs, and maintain strong performance?

1.5 Objectives

Based on the problem statement, the objectives of the project are divided into functional and non-functional objectives.

Functional Objectives

- Enable dynamic agent creation from declarative YAML or JSON configuration files.
- Provide a visual workflow builder for composing and managing multi-agent execution pipelines.
- Offer a unified interface for selecting models, prompts, schemas, providers, and tools without modifying the source code.
- Support reusable multi-agent pipelines that can combine specialized agents into structured workflows.
- Integrate SLM-evals results into the provider selection process to compare LLM and SLM options before deployment.

Non-functional Objectives

- Reduce vendor lock-in and operational cost by evaluating multiple providers, including proprietary LLM APIs and local SLM alternatives.
- Optimize deployment costs through containerized, configurable, and reusable agent services.
- Ensure observability and traceability by monitoring execution traces, token usage, latency, and estimated cost.
- Preserve modularity and scalability by allowing agents, tools, providers, and workflows to evolve independently.
- Strengthen security and provider independence by limiting hard-coded provider dependencies and supporting controlled configuration management.

1.6 Technology Choices

The project relies on a modular technology stack selected to support agent generation, orchestration, deployment, observability, and provider flexibility.

1.6.1 Development Frameworks and Dev Environment

- **FastAPI:** lightweight Python framework used to build high-performance asynchronous APIs for agents and platform services.



Figure 2: Logo of FastAPI

- **Next.js:** React-based framework used to build the web frontend and provide a modular interface for agents, workflows, and configurations.



Figure 3: Logo of Next.js

- **LangChain:** library used to construct LLM-driven agents, connect tools, and manage prompt-based application logic.



Figure 4: Logo of LangChain

- **LangGraph:** library used to define graph-based orchestration logic for stateful multi-agent workflows.

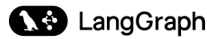


Figure 5: Logo of LangGraph

- **Jupyter Notebook:** interactive environment used during prototyping, experimentation, SLM-evals, and prompt engineering.



Figure 6: Logo of Jupyter Notebook

1.6.2 LLM and SLM Providers

- **OpenAI:** commercial LLM provider used to access GPT-series models for high-quality generation and evaluation tasks.



Figure 7: Logo of OpenAI

- **Groq:** commercial inference provider used to access fast LLM serving, including Llama-based models.



Figure 8: Logo of Groq

- **Microsoft Phi-4:** small language model considered for cost-efficient and potentially self-hosted inference.



Figure 9: Logo of Microsoft Phi-4

- **Meta Llama** : open-weight LLM used as an alternative provider option for flexible deployment and benchmarking.



Figure 10: Logo of Meta Llama

- **SLM-evals**: used to benchmark providers and models in order to choose the best mix for the VIP provider layer.

1.6.3 Database, Storage, and Experiment Tracking

- **PostgreSQL**: relational database used to store configurations, metadata, workflows, and other structured platform data.



Figure 11: Logo of PostgreSQL

- **MLflow**: experiment tracking and artifact registry used to version models, prompts, schemas, datasets, and evaluation metrics.



Figure 12: Logo of MLflow

1.6.4 Deployment and Containerization

- **Docker**: container platform used to package individual agents, the builder, the orchestrator, and supporting services into reproducible images.



Figure 13: Logo of Docker

- **Docker Compose**: used during development and testing to run multi-service setups with consistent networking and configuration.



Figure 14: Logo of Docker Compose

1.6.5 Observability and Monitoring

- **Laminar:** used to capture execution traces, token usage, latency, and cost metrics across agents.



Figure 15: Logo of Laminar

1.7 Methodology

1.7.1 Methodological Approaches Considered

Several methodological approaches were considered in order to structure the development of the VIP platform. The project combines software engineering, data science, agent lifecycle management, orchestration, deployment, and observability. Therefore, the selected methodology had to support both iterative development and long-term architectural control. The approaches considered were Scrum, IBM Data Science Methodology, and IBM Agent Development Lifecycle (ADLC).

1.7.2 Scrum Agile Methodology

Scrum is an Agile framework based on iterative and incremental development. The official Scrum Guide describes Scrum as an iterative and incremental approach used to optimize predictability and control risk. It supports progressive delivery by organizing work into short cycles, reviewing progress regularly, and adapting the next development steps according to feedback.

Scrum was considered because the project required frequent adjustments, progressive development, and regular validation of features such as agent creation, workflow orchestration, prompt management, and observability dashboards.

However, Scrum alone was not sufficient for this internship because the project required more than sprint-based feature delivery. It also required a structured lifecycle for AI/data-related work, architectural traceability, governance, deployment control, and continuous monitoring of AI agents. For this reason, Scrum-inspired practices were retained for implementation organization, but Scrum was not selected as the main project methodology.

1.7.3 IBM Data Science Methodology – The Foundational Methodology

The IBM Data Science Methodology was considered as the foundational methodology because it provides the macro lifecycle of the project. The CognitiveClass data science methodol-

ogy is structured around phases that move from business understanding and analytic approach to data requirements, collection, understanding, preparation, modeling, evaluation, deployment, and feedback. This structure makes it suitable for organizing an AI-oriented project from problem definition to operational feedback.

The methodology consists of 10 stages organized in a circular workflow:

- **Business Understanding:** defining the problem to be solved and establishing the main objectives.
- **Analytic Approach:** determining the appropriate analytical and technical approach to address the problem.
- **Data Requirements:** identifying the information and resources required to address the problem.
- **Data Collection:** gathering the required data and project inputs from the relevant sources.
- **Data Understanding:** exploring and examining the available information in order to understand quality, constraints, and relationships.
- **Data Preparation:** cleaning, transforming, and formatting the required data or resources for later use.
- **Modeling:** building predictive, descriptive, or intelligent models using appropriate algorithms and implementation choices.
- **Evaluation:** assessing the obtained results against the initial objectives and expected quality criteria.
- **Deployment:** implementing the solution in a production or operational environment.
- **Feedback:** monitoring performance, collecting feedback, and iterating on earlier stages when improvements are required.

Figure 16 presents the original IBM Data Science Methodology lifecycle, which was used as the main macro-framework for structuring the project from business understanding to deployment and feedback.

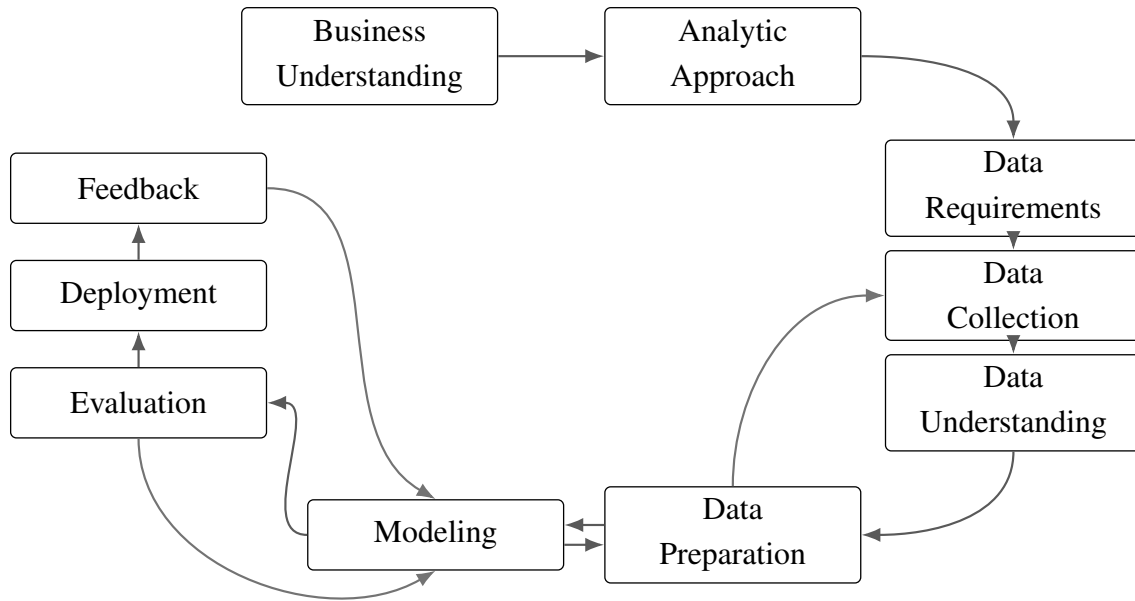


Figure 16: Original IBM Data Science Methodology lifecycle

This methodology is relevant for the VIP project because it supports a structured understanding of the business problem, the identification of requirements, the preparation of resources, the evaluation of outputs, and the deployment of the final solution. It also provides a feedback loop, which is important for improving agent behavior, model selection, and workflow reliability over time.

1.7.4 IBM Agent Development Lifecycle

While the IBM Data Science Methodology provides the overall project structure, it does not fully describe the specific lifecycle of AI agents. The VIP platform required a more agent-specific perspective because agents are generated from configuration, connected to tools, deployed as services, monitored, and improved after execution. For this reason, ADLC was used as a complementary framework.

IBM defines ADLC as a structured lifecycle for building, deploying, optimizing, and managing AI agents through distinct phases and iterative loops. In the context of VIP, this perspective is relevant because agents must remain reliable, observable, auditable, and aligned with business requirements after deployment.

In this report, the ADLC perspective is summarized through the following phases:

- **Plan:** alignment of stakeholders, use cases, objectives, success metrics, expected agent behavior, and operating procedures.
- **Code & Build:** implementation of agents through model selection, prompt design, tool integration, orchestration logic, version control, and controlled development environments.
- **Test & Release:** evaluation of agents against predefined criteria, including policy checks, security validation, and release readiness before cataloging or production use.

- **Deploy:** controlled rollout of validated agents into production environments with governance, versioning, rollback, and runtime security mechanisms.
- **Operate:** continuous observation and optimization of deployed agents using operational metrics such as accuracy, latency, cost, and user satisfaction.
- **Monitor:** ongoing auditing of agents and tools to support fairness, transparency, compliance, observability, and reproducibility.

Figure 17 illustrates the IBM Agent Development Lifecycle, which was used to guide the agent-specific phases of the project.

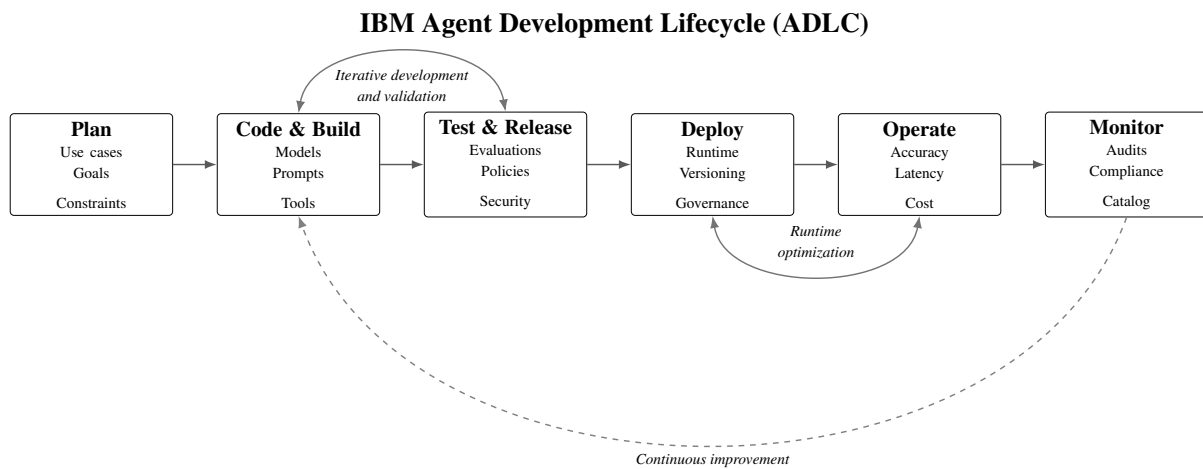


Figure 17: IBM Agent Development Lifecycle (ADLC)

In the VIP project, the Plan phase corresponds to defining agent use cases, expected behavior, schemas, and constraints. The Code & Build phase corresponds to configuring prompts, tools, models, and runtime artifacts. The Test & Release phase corresponds to validating agent outputs and schema compliance. The Deploy phase corresponds to packaging agents as Dockerized FastAPI services. The Operate and Monitor phases correspond to observing latency, token usage, cost, and execution traces.

IBM also describes agent building as the transformation of intent and design into concrete capabilities by configuring models, behavior, knowledge, and tools. This is aligned with the VIP approach, where agents are created from declarative configurations including prompts, schemas, tools, and model parameters.

1.7.5 Comparative Analysis of the Methodologies

The three methodologies were compared according to their relevance to the VIP project, especially with regard to flexibility, strategic vision, AI project support, agent lifecycle coverage, deployment, monitoring, and limitations.

Criterion	Scrum	IBM Data Science Methodology	IBM ADLC
Approach	Iterative and sprint-based	Structured and data/AI-oriented	Agent lifecycle-oriented
Flexibility	High	Medium to high	High
Strategic vision	Limited to product increments	Strong	Focused on agent operations
Fit for AI projects	Partial	Strong	Strong for agentic AI
Fit for agent lifecycle	Limited	Partial	Very strong
Deployment and monitoring support	Not detailed	Present through deployment and feedback	Central through deploy, operate, and monitor
Main limitation	Not enough governance for AI agents	Not specific enough for agent run-time behavior	Too agent-specific to structure the whole project alone

Table 1: Comparison of the methodological approaches considered for the VIP project

The comparison shows that none of the methodologies fully covers the needs of the VIP project when used alone. Scrum supports practical development organization, IBM Data Science Methodology provides the strategic project lifecycle, and ADLC addresses the agent-specific lifecycle required by configurable, deployable, and observable AI agents.

1.7.6 Adopted Hybrid Methodology for the VIP Project

Based on the comparison, the adopted methodology is a hybrid approach combining IBM Data Science Methodology and IBM ADLC, while keeping Scrum-inspired iterations for practical development organization.

IBM Data Science Methodology was selected as the main macro-framework because it structures the project from business understanding to deployment and feedback. It ensures that the project remains aligned with business objectives, technical constraints, and evaluation criteria.

ADLC was added because the VIP platform is centered on AI agents. The project required agent-specific phases such as planning agent behavior, configuring prompts and tools, validating outputs, deploying agents as services, operating them, and monitoring their execution.

Scrum was not selected as the main methodology because the project required stronger architectural governance and lifecycle control than a sprint-based framework alone could provide. However, Scrum-inspired practices were useful during implementation, especially for breaking the work into incremental tasks, validating features progressively, and adapting to feedback.

This hybrid methodology was therefore chosen because it provides a balance between structure, flexibility, and agent-specific lifecycle management. It supports the strategic organization of the project, the iterative improvement of AI components, and the operational requirements of an enterprise-grade agentic platform.

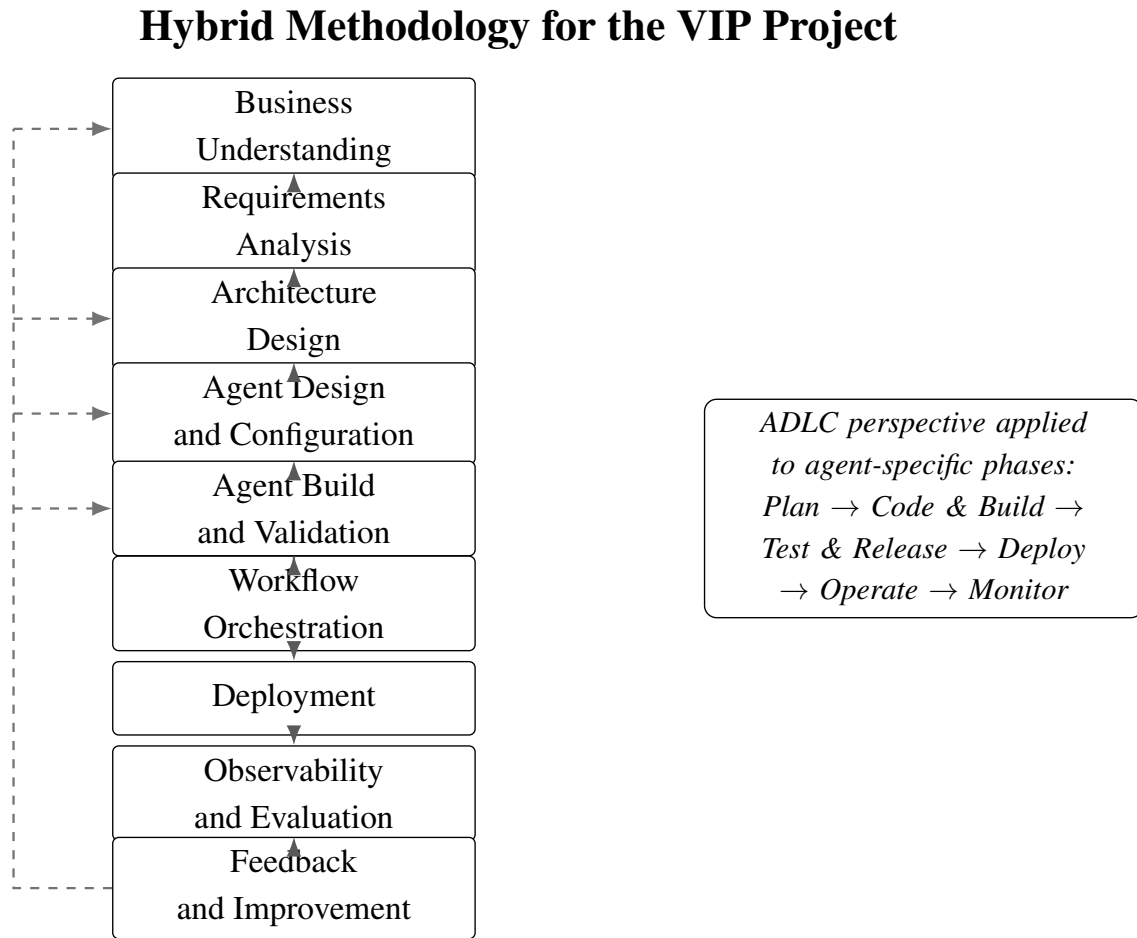


Figure 18: Hybrid methodology adopted for the VIP project

1.8 Project Timeline

The project officially started on December 1, 2025 and is scheduled to end on May 31, 2026, spanning a duration of six months. This timeframe was organized to support a structured and progressive approach, covering onboarding, research, design, development, integration, observability, and testing phases.

1.9 Summary

This chapter introduced the context of the internship by presenting the motivation behind enterprise AI agent orchestration, the host company Value Digital Services, and the objectives of the VIP project. It also clarified the main problem addressed by the project, namely the need for a modular, observable, cost-optimized, and provider-independent platform for creating and managing AI agents. The chapter then summarized the selected technologies, including FastAPI, Next.js, LangChain, LangGraph, Docker, MLflow, PostgreSQL, Laminar, and the LLM/SLM providers evaluated through SLM-evals, before presenting the methodological approach and the project timeline.

The next chapter focuses on the state of the art. It reviews existing agent frameworks, LLM providers, orchestration approaches, and evaluation methods in order to position the proposed

solution with respect to current tools and practices.

Chapter 2

State of the Art

This chapter presents the main concepts, tools, and evaluation approaches that form the state of the art for the VIP project. It reviews the evolution of agentic AI systems, examines representative orchestration and automation platforms, summarizes the main LLM and SLM provider options, and introduces the evaluation principles used to compare models in a reproducible way. The objective is not only to survey existing solutions, but also to identify the practical gaps that motivate the architecture of VIP.

2.1 AI Agents and Orchestration

AI agents extend the traditional prompt-response use of language models by combining instructions, tools, memory, and execution logic in order to pursue a task over multiple steps. In enterprise settings, the challenge is not only to generate responses, but to coordinate agents reliably, preserve state, validate exchanged data, and observe execution across workflows. For this reason, orchestration frameworks have become a central layer in modern agentic systems, providing reusable abstractions for tool usage, workflow control, persistence, tracing, and multi-agent collaboration. In the context of VIP, these frameworks are reviewed as design references rather than turnkey solutions, since the platform must remain configuration-driven, modular, and provider-independent.

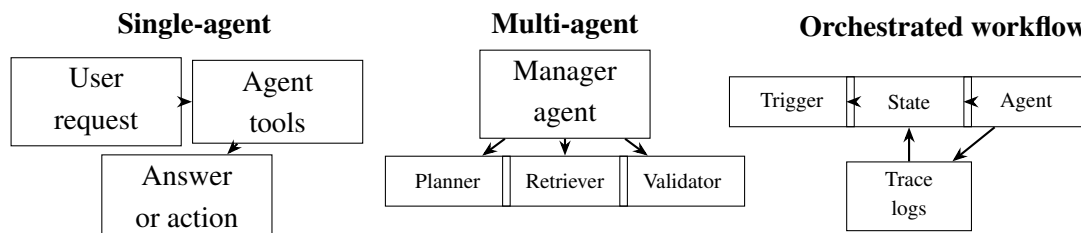


Figure 2.1: Orchestration patterns diagram—single-agent, multi-agent, and orchestrated workflow patterns.

2.1.1 Definitions

An AI agent is classically defined as a system that perceives its environment and acts to achieve objectives. In LLM-based applications, this idea becomes a model connected to instructions, tools, memory, control logic, and outputs that may affect external systems. The agent may execute a single role, such as retrieval or validation, or participate in a multi-agent

workflow where specialized components exchange intermediate results. The key distinction is that an agent follows a task across steps: it observes context, decides or delegates actions, uses tools, and adapts subsequent behavior based on results.

Synthesis. VIP treats agents as managed execution units rather than prompt wrappers because they require configuration, state, tools, permissions, and observable runtime behavior.

2.1.2 LLM and SLM Overview

Large language models (LLMs) provide broad reasoning, generation, and tool-use capabilities, but enterprise use often depends on remote provider APIs, network conditions, and external service availability. This dependence can increase latency variability, expose sensitive prompts or outputs to third-party infrastructure, and reduce operational control over deployment and versioning. Small language models (SLMs) offer a complementary option: they can be hosted closer to the application or inside controlled environments, which can improve privacy, latency control, and independence from a single provider. However, smaller models must be evaluated against concrete tasks to verify whether their quality is sufficient. VIP therefore links model selection to SLM-evals, which benchmarks LLM and SLM options under reproducible prompts, datasets, and evaluation criteria.

2.1.3 Motivation for Orchestration Frameworks

Raw LLM APIs produce model responses, but agentic applications need an execution layer around those responses. Developers must organize message passing between users, tools, and agents; preserve state across steps; apply retries when providers or tools fail; record tracing data for debugging and auditability; and coordinate several agents when work is delegated. Without a framework, these responsibilities become fragmented across application code, making behavior harder to test, reproduce, and monitor. Orchestration frameworks provide reusable patterns for graphs, loops, handoffs, memory, tool calls, and workflow control.

Synthesis. This motivates VIP’s orchestration layer, which coordinates generated agents while keeping execution observable and controllable.

2.1.4 Framework Survey

LangChain / LangGraph. LangChain / LangGraph is the LangChain ecosystem for model integration and graph-based agent orchestration. LangChain provides a standard model interface and agent abstractions, while LangGraph supplies durable execution, persistence, human-in-the-loop control, and stateful workflow graphs. It is strong at separating model calls, tools, state, and control flow. VIP learns from it by adopting provider-neutral model integration and explicit orchestration patterns for generated agents.

CrewAI. CrewAI is a framework for organizing autonomous agents into Crews and controlled execution paths into Flows. It is strong at role-based collaboration, task delegation, and stateful, event-driven workflow control where agents can exchange work through defined processes. VIP learns from it by separating collaborative agent roles from workflow governance, so planner, retriever, validator, and reporting agents can cooperate without turning coordination into unmanaged prompt logic.

OpenAI Agents SDK. The OpenAI Agents SDK is a production-oriented toolkit for defining agents, tools, handoffs, guardrails, sessions, and tracing around OpenAI model execution. It is strong at compact developer primitives for agent handoff, validation, persistent context, and trace visibility. VIP learns from it by treating handoffs, guardrails, sessions, and observability as first-class platform capabilities, while keeping the architecture independent from a single provider ecosystem.

Google ADK. Google ADK is a framework for building agents with tools, sessions, memory, evaluation, and workflow agents. It is strong at graph workflows and multi-agent workflow patterns, including sequential, parallel, and loop-style coordination. VIP learns from it by modeling generated agents as composable workflow nodes that can be routed, repeated, parallelized, observed, and evaluated rather than as isolated API endpoints.

Anthropic / MCP-oriented tooling. Anthropic / MCP-oriented tooling combines terminal-based agent work, Model Context Protocol connectivity, and multiple-agent or subagent patterns. It is strong at connecting agents to external tools through controlled interfaces while supporting practical software-engineering workflows. VIP learns from it by emphasizing scoped tool access, permission boundaries, and repeatable multi-agent patterns so agent execution remains governable in enterprise environments.

Synthesis. These frameworks show that VIP should combine configurable agent generation, explicit orchestration, provider flexibility, tool governance, state management, and observability in a single managed platform.

2.2 Comparison of Existing Platforms

Existing platforms solve different parts of the enterprise agent orchestration problem. Visual workflow platforms such as n8n simplify automation and integration, agent collaboration frameworks such as CrewAI make role-based decomposition easier to implement, and managed cloud platforms illustrate the value of an operational layer for execution and follow-up. However, these solutions do not provide the full combination required by VIP, namely configuration-driven agent generation, provider neutrality, reusable deployment units, observability, and evidence-based model selection. Their comparative study is therefore useful primarily to identify reusable patterns and architectural gaps.

n8n. n8n represents the visual workflow automation category. It models automations as node-based workflows and includes LangChain-oriented cluster nodes, where root nodes define main actions and sub-nodes provide connected capabilities such as models, memory, or tools. Its AI Agent node is explicitly tool-using and requires at least one tool sub-node, which makes it useful for connecting agent steps to external systems. VIP learns from n8n’s accessible workflow composition, but must add stronger agent generation, deployment, observability, and provider-selection control.

- **Relevance to VIP:** n8n inspired the visual workflow layer implemented in VIP, where generated agents can be composed into multi-agent execution pipelines rather than invoked only as isolated services.

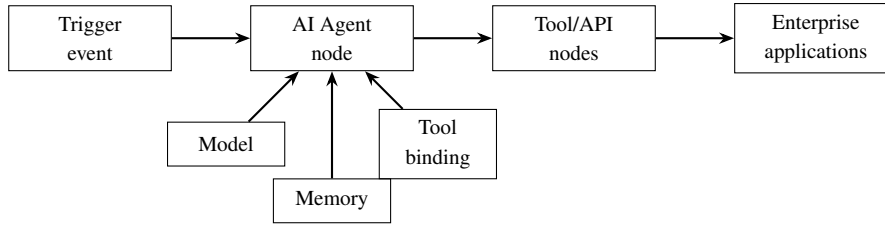


Figure 2.2: Visual workflow orchestration model—n8n-style node-based workflow.

CrewAI. CrewAI represents the agent collaboration category. Its documentation separates Crews, which are teams of autonomous agents, from Flows, which provide the stateful execution backbone for controlled workflows. This distinction is useful because enterprise agent systems need both role decomposition and process control. VIP learns from CrewAI’s planner-worker style collaboration, but requires a broader platform layer for generated services, centralized configuration, multi-provider support, and production monitoring.

- **Relevance to VIP:** CrewAI influenced VIP’s specialized agent roles, including planner, retriever, validator, and reporting agents coordinated through the orchestration layer.

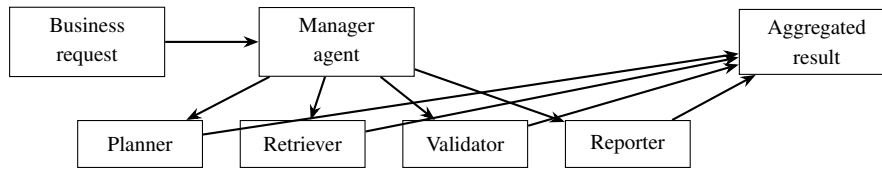


Figure 2.3: Role-based crew coordination—CrewAI-style manager/workers interaction.

Managed operational platform. Jarvio is kept as the representative managed-platform slot because its product page presents AI agents as operational assistants that can execute workflows, call APIs, schedule actions, and support business follow-up from a cloud environment. This category is strong at reducing infrastructure burden and making agent execution accessible to non-specialist users. VIP learns from the value of an operational layer, but must remain more portable, configurable, and provider-neutral than a domain-specific managed service.

- **Relevance to VIP:** Jarvio’s managed execution model reinforced VIP’s cost-aware evaluation work through SLM-evals and its cloud-neutral, provider-independent deployment strategy.

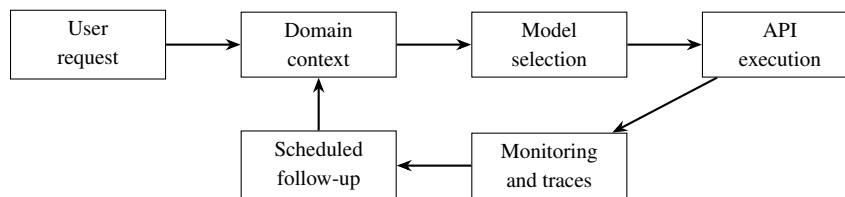


Figure 2.4: Managed operational platform lifecycle

Framework	Orchestration Model	Strengths	Weaknesses
n8n	Visual node-based workflow automation with AI Agent and LangChain cluster nodes	Accessible workflow design; broad integrations; practical connection between AI steps and external systems	Does not fully cover generated agent services, deployment units, governance, or model-selection evaluation
CrewAI	Crews for autonomous agent teams and Flows for stateful execution	Clear role decomposition; useful for planner-worker collaboration; strong reference for agent templates	More code-oriented; requires additional platform services for enterprise monitoring, configuration, and deployment
Jarvio	Managed cloud operational layer for AI-assisted business workflows	Practical operational experience; API execution and follow-up actions; reduced infrastructure burden	Vendor and domain dependency; less portable and less transparent for custom multi-provider architectures

Table 2: Comparison of existing platforms

2.3 LLM and SLM Providers

The provider layer is a strategic component of VIP because the choice of model directly affects latency, cost, deployment flexibility, and vendor dependence. Commercial APIs simplify integration and provide strong general-purpose performance, while smaller or open-weight models may become preferable when privacy, infrastructure control, or operational cost are stronger constraints. For this reason, the provider layer of VIP is designed to remain replaceable rather than hard-coded. The SLM-evals side project complements this design by benchmarking different providers under controlled prompts, datasets, and evaluation metrics before they are integrated into workflows.

Provider family	Integration mode	Strengths	Constraints
Commercial API providers	Remote API integration through provider SDKs or HTTP clients	Strong general-purpose capability; fast integration; managed model serving; frequent model updates	Provider dependence; external data processing; network latency; limited deployment control
Cloud-hosted open-weight models	Hosted inference through managed cloud or specialized inference platforms	More model choice; easier experimentation with open-weight families; scalable serving without local infrastructure	Still depends on third-party infrastructure; deployment and data-governance control vary by host
Self-hosted SLMs or open-weight models	Local, private-cloud, or containerized deployment near VIP services	Better privacy control; reduced external dependency; tunable deployment environment; useful for latency-sensitive workflows	Requires infrastructure management; model quality must be validated; operational monitoring remains necessary

Table 3: Provider comparison

2.4 Evaluation Framework

A key limitation of many current agent platforms is that model selection is often driven by vendor familiarity rather than controlled evaluation. In the VIP context, this issue is addressed through the SLM-evals pipeline, which combines reference-based similarity metrics, structural completeness checks, latency tracking, and LLM-as-a-judge scoring in order to compare providers on the same task under the same prompting conditions. The framework was later extended with layered evaluation, including RAGAS-based analysis and domain-specific numerical checks, to obtain a more robust comparison than lexical metrics alone. This evaluation block is essential because provider flexibility only has practical value when model choice is measurable and reproducible.

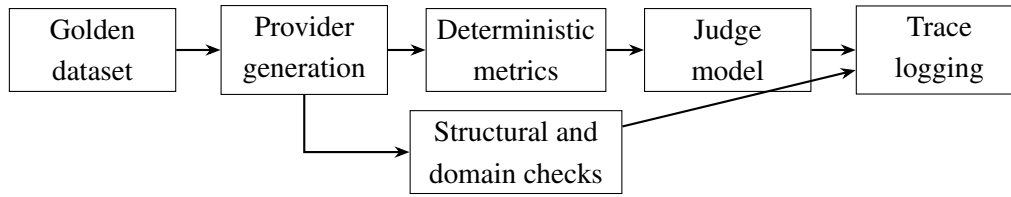


Figure 2.5: Evaluation pipeline—golden dataset, generation, metrics, judge, logging.

2.5 Gap Analysis and VIP Positioning

Existing tools show that the enterprise agent ecosystem is already mature in several focused areas. LangChain and LangGraph provide reusable abstractions for model calls, tools, graph-based workflows, persistence, and durable execution. In the platform comparison, CrewAI demonstrates practical role-based collaboration, n8n shows how visual workflow environments can make automation and integration accessible to non-specialist users, and Jarvio illustrates the value of a managed operational layer that handles execution and follow-up. These solutions therefore provide useful design references for VIP: they validate the importance of workflow modeling, tool connectivity, state management, and agent collaboration in enterprise use cases.

However, the same comparison also reveals important gaps. Most frameworks solve one part of the lifecycle without covering the full operational chain from agent definition to deployment, monitoring, and provider evaluation. Code-oriented frameworks can be powerful but remain difficult to expose directly to business users or to standardize across many generated agents. Visual platforms simplify workflow creation but do not fully address generated agent templates, versioned configuration, reusable deployment units, and deep runtime observability. Managed platforms reduce infrastructure effort, but usually introduce cloud dependency, limited portability, and weaker control over the provider layer. These limitations align with the problem statement of Chapter 1: VIP needs modularity, observability, deployment automation, multi-provider support, and cost-aware model choice rather than only isolated agent execution.

VIP is positioned at this intersection. Its objective is not to replace every agent framework with a new low-level library, but to build a controlled platform architecture around generated agents. In this architecture, agents are created from configuration, packaged as reusable services, connected through an orchestration layer, monitored during execution, and linked to a replaceable provider layer. SLM-evals strengthens this positioning by making model selection evidence-based rather than vendor-driven. The contribution of VIP is therefore architectural: it combines lessons from existing tools while adding the governance, observability, deployment, and evaluation mechanisms required for enterprise-grade agent operations.

- **Configurable agent generation:** VIP addresses the gap between code-heavy frameworks and reusable agent templates by generating agents from structured configuration.
- **Observability and governance:** VIP responds to weak runtime visibility by tracking execution traces, configuration artifacts, prompts, schemas, and operational metrics.
- **Provider-neutral model selection:** VIP reduces vendor dependence by keeping the provider layer replaceable and using SLM-evals to compare models under reproducible

conditions.

Capability	Existing tools	VIP response
Agent generation	Partial or	Configuration-
	code-heavy	driven
Orchestration	Workflow or	Reusable service
	agent-specific	workflows
Observability	Uneven across	Execution traces
	platforms	and metrics
Provider choice	Often tied to	Replaceable layer
	ecosystem	plus SLM-evals

Figure 2.6: VIP gap positioning matrix—existing tools vs VIP capabilities.

2.6 Summary and Transition

This chapter reviewed the technical and conceptual foundations needed to position VIP: agent definitions, orchestration patterns, representative platforms, provider trade-offs, and reproducible evaluation principles. The analysis showed that existing solutions are mature in specific areas, such as workflow automation, role-based collaboration, graph execution, and managed operations, but that they do not fully combine configuration-driven agent generation, reusable deployment, observability, governance, and provider-neutral model selection. These findings justify the architectural direction of VIP as a platform rather than a single isolated framework. The next chapter builds directly on this state of the art by presenting the proposed system architecture and explaining how the identified requirements are translated into concrete components, including the agent generator, orchestration layer, provider abstraction, observability services, and evaluation workflow.